

Memory Layout & Cache

Adam HAMOUCHE

May 2026

1. Mémoire et hiérarchie cache

La RAM est un tableau d'octets, chaque octet a une **adresse unique**. Le CPU y lit des variables en cherchant l'adresse correspondante. Problème : la RAM est **lente** (100ns, 300 cycles perdus). Solution : le **cache**, une mémoire intermédiaire très rapide entre le CPU et la RAM.

Niveau	Taille	Latence	Scope
L1 cache	~32 KB	~1 ns	1 par cœur
L2 cache	~256 KB	~4 ns	1 par cœur
L3 cache	~8 MB	~12 ns	Partagé entre cœurs
RAM	Plusieurs GB	~100 ns	Globale

Cache hit : donnée trouvée dans le cache → accès quasi instantané.

Cache miss : donnée absente → chargement depuis la RAM → 300 cycles perdus.

Quand le CPU charge une donnée, il ne charge pas juste l'octet demandé : il charge un bloc de **64 octets contigus** autour de l'adresse — c'est une **cache line**. Le CPU parie que les données voisines seront lues ensuite (*localité spatiale*).

```
// Structure contigus (vector) : 1 cache miss puis acces gratuits
int arr[1000];
for (int i = 0; i < 1000; i++) arr[i] *= 2;
// 1er acces : cache miss => charge 64 octets => 16 ints deja en cache

// Structure non contigus (list) : cache miss potentiel a chaque iteration
Node* n = head;
while (n) { n->val *= 2; n = n->next; } // chaque noeud = adresse differente
```

2. Stack vs Heap

Deux zones mémoire distinctes pour les allocations :

Stack : alloc/dealloc automatique, variables locales, taille limitée (8 MB), ultra rapide (déplacement de pointeur), lifetime liée au scope.

Heap : alloc via `new`/smart pointers, objets dont la vie dépasse le scope, taille limitée par la RAM, plus lent (appel système, fragmentation).

```
void foo() {
    int x = 42; // Stack : detruit a la fin de foo()
    std::array<float, 16> m; // Stack : taille connue compile-time

    auto tex = std::make_unique<Texture>("rock.png"); // Heap : survit au scope
    std::vector<Vertex> verts; // Heap : taille dynamique
} // x, m detruits automatiquement. tex detruit via unique_ptr (RAII).
```

En engine dev : préférer la stack pour les petites données temporaires (matrices de transformation, vecteurs locaux). Le heap pour les ressources (textures, meshes, buffers GPU) dont la durée de vie dépasse le scope d'une fonction.

3. Alignement mémoire

Le CPU lit les données efficacement uniquement si elles sont placées à des adresses **multiples de leur taille**. Un mauvais alignement force le CPU à effectuer **deux lectures** au lieu d'une.

Le compilateur insère automatiquement des **octets de padding** pour corriger l'alignement — mais cela augmente le **sizeof** et la quantité de mémoire chargée en cache.

```
struct Mauvais {
    char    a;        // 1 octet   | offset 0
    // 7 octets de padding
    double  x;        // 8 octets  | offset 8
    char    type;     // 1 octet   | offset 16
    // 3 octets de padding
    float   health;   // 4 octets  | offset 20
}; // sizeof = 24 octets, seulement 14 octets utiles

struct Bon {
    double  x;        // 8 octets  | offset 0
    float   health;   // 4 octets  | offset 8
    char    a;        // 1 octet   | offset 12
    char    type;     // 1 octet   | offset 13
    // 2 octets de padding
}; // sizeof = 16 octets      8 octets economises par instance
```

Règle pratique : grouper les membres par taille (8 octets, puis 4, puis 2, puis 1). Utiliser `static_assert(sizeof(T) == N)` pour vérifier.

4. AoS vs SoA

AoS (Array of Structures) : tableau de structs. Naturel, mais charge toutes les données d'une entité même si le système n'en utilise qu'une partie.

SoA (Structure of Arrays) : une struct contenant un tableau par champ. Charge uniquement les données nécessaires au système.

```
// AoS : cache line charge position + velocity + health + alive
// meme si le systeme n'utilise que position et velocity
struct Entity { vec3 position; vec3 velocity; float health; bool alive; };
Entity entities[10000];
for (auto& e : entities) e.position += e.velocity; // donnees inutiles chargees

// SoA : cache line charge uniquement les donnees du systeme courant
struct EntityStorage {
    vec3  positions[10000];
    vec3  velocities[10000];
    float healths[10000];
    bool  alives[10000];
};
for (int i = 0; i < 10000; i++)
    positions[i] += velocities[i]; // 100% des donnees chargees sont utilisees
```

5. False Sharing

Deux threads écrivent des variables **différentes** mais dans la **même cache line** (64 octets). Chaque écriture invalide la cache line pour l'autre thread → cache miss en boucle malgré l'absence de vraie collision.

Solution : forcer chaque variable dans sa propre cache line avec `alignas(64)`.

À utiliser uniquement si : plusieurs threads **écrivent** des variables distinctes dans la même cache line.

```
// Probleme : renderCount et audioCount dans la meme cache line
struct Counters {
    int renderCount; // offset 0
    int audioCount;  // offset 4 -- meme cache line !
};
// Thread rendu ecrit renderCount => invalide la cache line pour le thread audio
// Thread audio doit recharger => cache miss => invalide pour le thread rendu...

// Solution : une cache line par variable
```

```
struct Counters {  
    alignas(64) std::atomic<int> renderCount; // cache line 0  
    alignas(64) std::atomic<int> audioCount;  // cache line 1  
};  
  
// Unreal  
alignas(CACHE_LINE_SIZE) TAtomic<int32> RenderCount;
```